

Optimized Resource Allocation in Vehicle Edge Computing Through Platoon Collaboration

Liang Zhao¹, Member, IEEE, Yuhang Feng, Ammar Hawbani², Lexi Xu³, Senior Member, IEEE, Zhi Liu⁴, Senior Member, IEEE, and Yuanguo Bi⁵, Member, IEEE

Abstract—In modern vehicular networks, the absence of infrastructure support, such as roadside units (RSUs), presents significant challenges for efficient task offloading and allocation. Limited computational capabilities of individual vehicles, combined with task allocation imbalances caused by varying task complexity and vehicle capacities, further complicate the process. Additionally, the formation of vehicular platoons requires accurate future route and destination information to ensure stable collaboration and effective coordination. However, such information can be challenging to obtain due to dynamic and unpredictable road environments, hindering the reliability of platoon formation. To address these challenges, we propose a platoon-based offloading strategy that integrates deep reinforcement learning (DRL) and long short-term memory (LSTM) networks to enhance task allocation efficiency. This approach also leverages the convoy formation algorithm considering future positions (CFA-FPs) to manage platoon constraints effectively. Experimental results demonstrate that our method significantly improves key performance metrics, including total computation cost, latency, and offloading success rate, compared to other task offloading strategies.

Index Terms—Deep reinforcement learning (DRL), platoon, task offloading, vehicle edge computing.

I. INTRODUCTION

NOWADAYS, with the proliferation of connected and autonomous vehicles, vehicular applications face the challenge of processing complex and diverse data while imposing severe requirements on cloud-based networks. In particular, long-distance communication between vehicles and cloud servers significantly impacts latency-sensitive tasks [1].

Received 17 December 2024; accepted 13 January 2025. Date of publication 16 January 2025; date of current version 23 May 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62372310 and Grant W2421018; in part by the Liaoning Province Applied Basic Research Program under Grant 2023JH2/101300194; in part by the Liaoning Revitalization Talents Program under Grant XLYC2203151; and in part by THE ROYAL SOCIETY-International Exchanges 2021 Cost Share (NSFC) under Grant IEC\NSFC\211034. (Corresponding authors: Ammar Hawbani; Lexi Xu.)

Liang Zhao, Yuhang Feng, and Ammar Hawbani are with the School of Computer Science, Shenyang Aerospace University, Shenyang 110136, China (e-mail: lzhao@sau.edu.cn; 15840378600@163.com; anmande@ustc.edu.cn).

Lexi Xu is with the Research Institute, China Unicom (China United Network Communications Corporation), Beijing 100048, China (e-mail: xulx29@chinaunicom.cn).

Zhi Liu is with the Department of Computer and Network Engineering, The University of Electro-Communications, Chofu 182-8585, Japan (e-mail: liu@ieee.org).

Yuanguo Bi is with the School of Computer Science and Engineering, Northeastern University, Shenyang 110819, China (e-mail: biyuanguo@mail.neu.edu.cn).

Digital Object Identifier 10.1109/IJOT.2025.3530573

With the rapid advancement of intelligent transportation systems, ensuring low latency, high reliability, and sufficient bandwidth has become critical for maintaining network performance. In such systems, addressing the growing computational demands of vehicles is a significant challenge, particularly in scenarios where vehicles rely on their own or nearby resources for task processing. Efficient task offloading mechanisms and resource coordination are essential for reducing communication delays and enhancing the overall responsiveness and efficiency of the system. Collaborative approaches that leverage local vehicular resources offer promising solutions to these challenges by alleviating computational burdens and improving task allocation efficiency [2].

However, the conventional method for offloading tasks from vehicles to roadside units (V2R) can lead to increased delays and higher energy usage [3]. For instance, when many tasks are offloaded, the restricted computing capacity of the RSU can become overwhelmed, causing further delays in task processing [4]. This situation becomes especially pronounced in high-density traffic or complex environments, significantly affecting the performance of vehicular applications. On the other hand, given the increasing computational resources equipped in vehicles [5], vehicles can be viewed as distributed computing systems. Therefore, resource-abundant vehicles can offload tasks to available peer vehicles through vehicle-to-vehicle (V2V) communication, effectively utilizing computing resources [6]. This strategy not only balances the load among vehicles but also enhances the overall resource utilization of the network.

Despite advancements in task-offloading systems using RSUs, new challenges arise for vehicle task-offloading in regions lacking RSU coverage. Particularly in rural or remote areas, deploying RSUs is often insufficient, leading to situations where vehicles cannot rely on external infrastructure for processing computational tasks. Thus, direct communication and collaboration between vehicles become crucial.

To address the task offloading challenges in the RSU-free scenario mentioned above, this article proposes the following scenarios and solutions. In the RSU-free context, the platoon collaboratively offloads peripheral tasks. To achieve this requirement, we first consider scenarios without RSUs and cases where each vehicle has limited computing power, addressing task offloading and resource allocation issues within a platoon. Second, varying computing power among vehicles and differing task loads can lead to imbalanced task allocation. To tackle this issue and enhance task offloading

efficiency, we combine deep reinforcement learning (DRL) with long-short-term memory (LSTM) networks. Additionally, we employ an optimized K -means algorithm to manage vehicles lacking future path and destination information, addressing another source of scene instability. The main contributions of this article are as follows.

- 1) We propose a platoon-based offloading scheme that leverages collaborative offloading of a platoon and with nearby vehicles to manage offloading tasks in scenarios without RSUs efficiently and where the computing power of individual vehicles is limited.
- 2) We improve the offloading strategy when applying DRL by combining DRL with LSTM by predicting the state information to improve performance regarding load conditions and latency.
- 3) We propose the convoy formation algorithm considering future position (CFA-FP) formation algorithm to solve the platoon formation problem when future path and destination information are unknown. The advantage of this algorithm is that it allows the formation to be carried out based on the vehicle's predicted future positions, even when such information is unavailable.

The remainder of this article is organized as follows. Section II presents the related work, Section III presents the system model, Section IV outlines the solution, Section V analyzes the experimental results, and Section VI provides the conclusion.

II. RELATED WORK

In recent years, RSU-based vehicle task offloading schemes have received significant attention and research. Ouyang [7] proposed a computing resource optimization allocation scheme based on RSU base stations equipped with multiple servers, enabling vehicles to flexibly choose between local execution or offloading to base stations/RSUs. Zhan et al. [8] divided the road according to the service range of the RSU and proposed a queuing optimization scheme to optimize the long-term costs for vehicles with insufficient computing power. Li et al. [9] introduced a location-based task offloading method that allows vehicles to send tasks to the nearest RSU for either solo processing or joint processing with other RSUs. Peng and Shen [10] investigated the resource allocation issue by examining edge nodes and macro-cell base stations equipped with RSUs to enhance user Quality of Service (QoS).

Currently, cellular vehicle-to-everything (C-V2X) communication modes employ Mode 3 or Mode 4 to adapt to different scenarios. In scenarios lacking RSUs, RSU-based schemes become inapplicable, necessitating task offloading between vehicles to address these challenges. Many studies have sought to generalize the single-user task offloading problem to multiuser situations, as multiuser environments are common in real-world applications. Similar to single-user offloading, the effectiveness of multiuser offloading algorithms is typically assessed using metrics, such as energy consumption, computing power, and resource efficiency. For example, a dynamic task offloading strategy based on noncooperative game theory was proposed in [11], significantly improving response time

and task failure rates (MDPI). Research indicates that optimizing resource management and minimizing delays through dynamic load balancing techniques is effective [12]; real-time dynamic task allocation can maximize the operational efficiency of the platoon, while collaborative strategies can enhance computational efficiency and resource utilization [13].

This study adopts C-V2X Mode 4, which enables distributed, self-organized resource allocation, allowing vehicles to independently select and reserve resources without cellular infrastructure. Distributed resource allocation is facilitated through orthogonal frequency division multiplexing (OFDM) to achieve high-spectral efficiency and minimize channel interference, making it particularly suited to dynamic and dense traffic environments [14]. In this mode, distributed learning algorithms, such as Q -learning, are employed for power and subchannel selection, enabling vehicles to adaptively allocate communication resources. A Markov model is utilized for resource reservation in platooning, ensuring stable intraplatoon communication and reducing channel collisions in high-traffic conditions without centralized coordination. Additionally, vehicles can implement multihop transmission and topology-aware techniques for reliable long-range communication within a platoon [15]. This distributed, adaptive resource allocation approach enhances C-V2X scalability and communication efficiency for multiplatoon collaboration, achieving effective resource management in high-density scenarios. To address communication challenges in shared channels without centralized coordination, Molina-Masegosa et al. proposed a Geo-based scheduling method for C-V2X networks. This approach leverages geographical location and vehicle ordering to enable deterministic subchannel selection, reducing packet collisions and mitigating the hidden terminal problem [16].

In a recent study, Deng et al. [17] introduced an autonomous partial offloading system specifically designed for delay-sensitive tasks in multiuser environments. This system employs reinforcement learning-driven offloading algorithms to reduce task execution delays. Elgendy et al. [18] proposed an offloading model in multitask, multiuser environments where computing tasks are assigned to edge servers. Chen et al. [19] examined the impact of data transmission on system performance and proposed storing computational tasks on servers to reduce overhead. Chen et al. [20] explored dynamic task distribution methods in NOMA networks and proposed a collaborative multiagent DRL model to improve cost-effectiveness and reduce latency in computing resources. Peng et al. [21] developed a tailored offloading model to minimize latency and energy consumption, targeting device-to-device systems with variable channel conditions and unpredictable application demands.

Furthermore, Virdis et al. [22] developed a platoon system based on mobile edge computing that allows vehicles to periodically report their speed and location information. The system uses a formation algorithm to organize vehicles into platoons and sets the acceleration of platoon vehicles through a coordination control algorithm to maintain a safe distance. Yang et al. [23] proposed a contract-based incentive system to encourage vehicles traveling in the same direction to form platoons, considering their computing power as a

key factor. Heinovski and Dressler [24] proposed a platoon formation method using expected driving speed as the main matching criterion. Wang et al. [25] explored cooperative formation across multiple lanes and introduced a two-stage resource allocation approach to improve delay performance. Mahbub and Malikopoulos [26] introduced a platoon formation method for mixed traffic involving both autonomous and human-driven vehicles, where the autonomous vehicle, as the leader, guides the human-driven vehicles to form a platoon. However, most existing studies rely on information obtained from future vehicle paths and destinations, which become inapplicable when such information is not available.

In summary, while RSU-based and edge-computing solutions offer various strategies for resource allocation and task offloading, they often rely on centralized infrastructure or assume specific path information. In contrast, this study focuses on decentralized, platoon-based task offloading, leveraging C-V2X Mode 4 for distributed, self-organized resource allocation without RSU support. By integrating dynamic communication scheduling and adaptive channel management, our approach addresses the limitations of traditional methods, enhancing scalability and efficiency in dense, high-traffic environments.

III. SYSTEM MODEL AND PRELIMINARY

This section presents the mobility model, task model, average latency, and problem formulation [27]. Table I lists the notation and definitions. The platoon is used as the basic allocation unit for task offloading.

This section introduces our proposed MML-DDPG-based vehicle decision-making framework, which aims to optimize the computational resource allocation and management during task offloading.

Fig. 1 illustrates the platoon-based task offloading scenario, encompassing platoon formation, resource allocation, and task offloading processes. Vehicles are grouped into platoons with designated cluster heads (in red) that manage intraplatoon coordination and resource distribution. Task vehicles offload their computation to computing vehicles within the platoons, shown by black arrows for task transfers and green arrows for service requests. This setup highlights the dynamic interaction among vehicles, with cluster heads coordinating task assignments and ensuring efficient resource allocation within each platoon.

A. Mobility Model

Each vehicle V_n has three main attributes: location (loc_n), speed (v_n), and direction (dr_n), where v_n is a scalar and $dr_n \in [-1, 1]$ represents the steering angle. Based on the free-flow traffic model and existing literature, we assume vehicles travel at a constant speed, which is Gaussian distributed. Mean velocity $\bar{v}$ and its variance σ_v are expressed by the formulas

$$\bar{v} = v_{\max} + \left(\frac{N}{1 - N_{\max}} \right), \sigma_v = \alpha_v \bar{v} \quad (1)$$

here, α_v is a scaling parameter, N represents the vehicle count, and $N_{\max}$ denotes the maximum vehicle capacity of the road.

TABLE I
KEY NOTATIONS LIST

Symbol	Description
V_1	Intelligent vehicle
N	Set of intelligent vehicle
loc_n	Location of the vehicle
dr_n	Direction of vehicle
v_n	Speed of the vehicle
I_k	Computational task
I	Set of tasks
$I_k^{(s)}$	The sth sub-task of task I_k
Mq	The message queue
σ_k	Download size of task I_k
γ_k	Size of upload data for task I_k
δ_k	Task I_k completion time
P_i	Priority of the i^{th} subtask
LA^{trans}	Latency of data transmission for the i^{th} subtask
LA^{comp}	Latency of computation for the i^{th} device
En^{trans}	The energy consumed during transmission
En^{comp}	The energy used for computing
LS	The comprehensive load status

The velocity V_n of vehicle v_n at time t is distributed as $v_n \sim (\bar{v}, \sigma_v)$, based on its position from the previous time slot $t-1$

$$loc_n(t) = dr_n(t-1) \cdot v_n(t-1) + loc_n(t-1). \quad (2)$$

B. Task Model

Tasks are characterized by attributes like download size σ , task completion time δ , computational requirement cr , and upload size γ . These attributes are provided on request by the customer's vehicle. For the upload size γ , each task I is related to the download size σ via the parameter β , so $\gamma = \beta\sigma$. Thus, the profile I_k of any task can be represented as $\{I_k^{(1)}, I_k^{(2)}, \dots, I_k^{(s)}\}$, where s is the maximum number of subtasks. Although each subtask transfers the same amount of data, they require different amounts of computing power. The algorithm ensures that the total of the computational requirements of every subtask is not less than that of the original task, that is, $\sum I_k^{(s)} cr_k^{(s)} \geq cr_k$. Considering economic factors, when a vehicle V_n offloads a task, it also sets the service price for the resource provider. The payment amount c_n is determined by the amount of the task, that is, offloaded.

To enhance the model's prediction capability for future scenarios, we introduce a time series-based prediction framework to estimate the future position of the vehicle, modeled as in (3), where $\mathcal{F}$ is based on the historical position data $\{loc_n(t-i)\}_{i=0}^m$ forecast function, including m on behalf of considering the history of the time window size, and k is to predict the future location of the time step

$$\hat{loc}_n(t+k) = \mathcal{F}(\{loc_n(t-i)\}_{i=0}^m). \quad (3)$$

In addition, considering the randomness and uncertainty in real traffic environments, we introduce random disturbances into the model of the vehicle speed and direction. Therefore, the speed and direction updates of the vehicle are given by (4), where ϵ_v and ϵ_{dr} are random perturbations of velocity and direction, respectively, which are usually assumed to

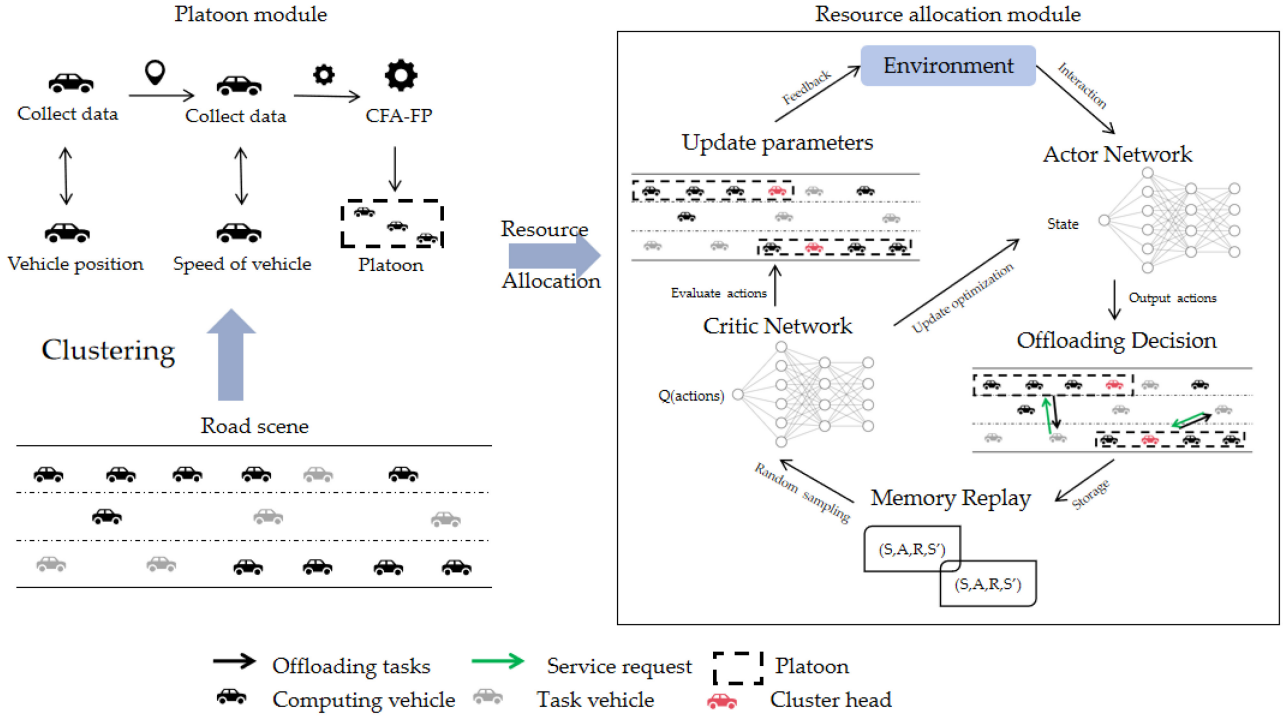


Fig. 1. System overview of platoon collaboration and task offloading.

follow a specific probability distribution, such as the normal distribution:

$$v_n(t) = \epsilon_v + \bar{v}, \quad dr_n(t) = \epsilon_{dr} + dr_n(t-1). \quad (4)$$

To manage task priorities, the analytic hierarchy process (AHP) is employed to assess task rankings [28]. The AHP effectively addresses priority-based scheduling issues and is used across various fields, as detailed in [29]. Let $Mq = \{mq_1, mq_2, \dots, mq_j\}$ denote the set of messages the mobile device sends within the time frame from 0 to t . In our study, task priority is determined by three key attributes: 1) task completion time; 2) CPU cycles; and 3) data size.

To implement AHP, a pairwise comparison matrix is constructed, where the relative importance between attributes is compared. Specifically, task completion time is assigned the highest priority, followed by CPU cycles, and then data size, indicating their relative importance for task scheduling. This prioritization is reflected in the normalized weight vector $W = [TaskCompletionTime, CPUCycles, DataSize]$, derived by averaging the normalized pairwise comparisons. The corresponding vector of task the i th is defined as $f^r = [TaskCompletionTime, CPUCycles, DataSize]$. By leveraging AHP, the system effectively quantifies task priorities and ensures tasks are scheduled based on a clear and systematic evaluation of these three attributes.

First, the AHP matrix $A = (a_{ij})_{3 \times 3}$ is constructed by comparing factors within the same level

$$a_{ij} = \frac{1}{a_{ji}} \quad (5)$$

where a_{ij} indicates the relative importance between attribute i and j .

Next, we construct the weight matrix $\Delta = (u_r^k)_{J \times 3}$, with J indicating the number of tasks and u_r^k denoting the significance of the RTH task with respect to the KTH criterion

$$U_r^k = \frac{\sum_{j=1}^k f_j^r}{\sum_{i=1}^3 \sum_{j=1}^3 a_{ij}}. \quad (6)$$

In conclusion, the prioritization vector PV for the set of tasks is established

$$PV = \Delta \times \Lambda = \begin{bmatrix} u_1^1 & u_1^2 & u_1^3 \\ u_2^1 & u_2^2 & u_2^3 \\ \vdots & \vdots & \vdots \\ u_j^1 & u_j^2 & u_j^3 \end{bmatrix} \cdot [\lambda_1, \lambda_2, \lambda_3]^T \quad (7)$$

here, Λ denotes the eigenvalues related to the factor weights in the AHP matrix. Every component of the PV vector reflects the relative importance of the corresponding task.

C. Average Latency

Next, we focus on the latency problem within the subtask dependency model. The average latency LA^{avg} in mobile applications includes: 1) the time LA^{trans} required for data transfer between subtasks and 2) the processing time LA^{comp} . Given that data transfer amounts and bandwidth fluctuate, we define a general transmission delay for the i th subtask to download and upload data within Δt

$$\begin{cases} LA_i^{\text{dow}}(t_0) = \int_{t_0}^{t_0+\Delta t} \frac{c_i^{\text{in}}(t)}{N_i^{\text{dow}}(t)} dt \\ LA_i^{\text{up}}(t_0) = \int_{t_0}^{t_0+\Delta t} \frac{c_i^{\text{out}}(t)}{N_i^{\text{up}}(t)} dt \end{cases} \quad (8)$$

where N_i^{dow} denotes the downlink capacity for the i th subtask, while N_i^{up} denotes the uplink bandwidth for the same subtask

$$\text{LA}_i^{\text{trans}}(t_0) = x_{\text{dow}}\text{LA}_i^{\text{dow}}(t_0) + x_{\text{up}}\text{LA}_i^{\text{up}}(t_0) \quad (9)$$

where x_{dow} signifies if the subtask requires downloading data from the network and x_{up} signifies if the result needs to be uploaded. Based on the DAG matrix L , these values establish data dependencies. x_{dow} equals 1 if the current subtask runs on a different edge server than its predecessor. Likewise, x_{up} equals 1 if the current subtask and its subsequent ones must be deployed to different edge servers. In all other cases, these values are defined as 0, that is, $x_{\text{up}} = 0$ and $x_{\text{dow}} = 0$.

Furthermore, the computational latency of the i th subtask to process the data in Δt time is specified as

$$\begin{aligned} \text{LA}_i^{\text{comp}}(t_0) = & x_{\text{off}} \int_{t_0}^{t_0+\Delta t} \frac{c_i^{\text{in}}(t)}{f^{\text{server}}(t)} dt \\ & + (1 - x_{\text{off}}) \int_{t_0}^{t_0+\Delta t} \frac{c_i^{\text{in}}(t)}{f^{\text{local}}(t)} dt \end{aligned} \quad (10)$$

where $f^{\text{server}}(t)$ denotes the processing rate for the i th subtask and $f^{\text{local}}(t)$ denotes the rate at which the local device processes the subtask, while x_{off} indicates whether the subtask should be transferred to an edge server.

Finally, the mean delay for all mobile tasks over Δt can be computed as

$$\text{LA}^{\text{avg}}(t_0) = \frac{\sum_{j=1}^{AN} \sum_{i=1}^{TN_j} (\text{LA}_i^{\text{trans}}(t_0) + \text{LA}_i^{\text{comp}}(t_0))}{AN} \quad (11)$$

where AN represents the total task count and TN_j indicates the number of subtasks into which each application is divided. $\text{LA}_i^{\text{trans}}(t_0)$ and $\text{LA}_i^{\text{comp}}(t_0)$ calculated, respectively, using (10) and (11). This approach not only helps to monitor and manage the efficiency of data processing tasks but also provides a practical tool to optimize the performance of the overall system.

By assessing each computing device's load status to evaluate cluster utilization, we can prevent server overload and boost the network's data processing capacity. A smaller load value indicates a better load state. In t_0 moment, I th all resources in a computing device load, $\text{Load}_i(t_0)$

$$\text{Load}_i(t_0) = \int_{t_0}^{t_0+\Delta t} \left(\sum_{k=1}^{ls} U_k \times L_k(t) dt \right) \quad (12)$$

where ls denotes the count of resource metrics used for load state calculation, U_k is the assigned weight for the k th resource, and $L_k(t)$ indicates the utilization of the k th resource per unit time. Next, we apply the Root Mean Square Error method to determine the overall load state of the cluster $LS(t_0)$

$$LS(t_0) = \sqrt{\frac{\sum_{i=1}^{1+m+n} (\text{Load}_i(t_0) - \text{Load}_i(t_0))^2}{1+m+n}}. \quad (13)$$

IV. PROPOSED SOLUTION

Into two segments is the problem broken down, to better realize resource allocation and task offloading in road scenarios with unstable platoon factors. First, the improved CFA-FP

algorithm was used for vehicle formation to form a stable and efficient platoon. The algorithm not only considers the position, speed, and direction of vehicles but also pays special attention to the impact of vehicles entering and exiting on the stability of the platoon. Second, to effectively manage the platoon and optimize communication, an optimized deep deterministic policy gradient (DDPG) reinforcement learning algorithm is implemented to enhance the flexibility and adaptability of platoon decision-making to cope with the challenges brought by dynamic changes of vehicles, and resource allocation and task offloading is carried out to ensure efficient and fair utilization of resources in the dynamic changing vehicle environment.

A. Problem Formulation

The computational cost of task offloading among vehicle platoons can be denoted by the symbol Ω , defined as

$$(P) : \min \Omega = \sum_{i \in I} (\alpha(\text{LA}_i^{\text{trans}} + \text{LA}_i^{\text{comp}}) + \beta \eta c_n) \quad (14)$$

where α and β are weights for the delay and service price, respectively, within $[0, 1]$ and adjustable based on the context, with η serving as a parameter. The goal is to minimize Ω , defined as

$$\text{s.t. } V_n \subseteq V_N \quad (15)$$

$$(\text{LA}_i^{\text{trans}} + \text{LA}_i^{\text{comp}}) \leq T_i \quad \forall i \in I \quad (16)$$

$$\rho_i \in [0, 1] \quad \forall i \in I \quad (17)$$

$$0 \leq (f'_v \mid \rho_i \neq 0) \leq f_v \quad \forall k \in I \quad (18)$$

$$\sum_{k=1}^i (f'_v \mid \rho_k \neq 0) \leq f_v \quad \forall k \in I. \quad (19)$$

Condition (15) states that the offloaded vehicles must be service vehicles within the road scenario to assist with computation. Condition (16) ensures that the total delay remains within the maximum allowed time. Condition (17) mandates that tasks may be offloaded to vehicles for computation within a specified proportion. Condition (18) specifies that the resources assigned to the task on the vehicle must be nonnegative. Condition (19) specifies that the total resources allocated must not surpass the overall computational capacity.

The difficulty of finding the optimal solution increases exponentially with respect to the number of transmitting vehicles (TVs) when ρ_n is 0 or 1, making it NP-hard [30]. This complexity worsens with the introduction of partial offloading, where ρ_n can vary between 0 and 1. Consequently, we tackle this issue in the following section using DRL.

Our platoon collaboration strategy employs a layered approach to communication management, minimizing overhead and ensuring efficiency in high-traffic environments. We implement a communication scheduling mechanism based on vehicle density and task priority, dynamically adjusting transmission intervals to reduce unnecessary data exchange by prioritizing essential information. Platoon leaders aggregate and compress data from intraplatoon vehicles, reducing communication load and redundancy, and enhancing scalability in dense traffic. Tasks are prioritized by urgency, allowing

only critical updates to be transmitted in real-time, while nonessential data is shared periodically to reduce channel congestion.

We utilize an adaptive C-V2X channel assignment mechanism to balance load across multiple channels, preventing congestion and improving reliability. The hierarchical communication structure enables platoon leaders to coordinate internal tasks while broadcasting only necessary information to other platoons. Coupled with decentralized learning on each vehicle, this setup allows for autonomous decision-making, further minimizing direct vehicle-to-vehicle communication. Additionally, local processing resources are utilized to handle data closer to the vehicles, reducing reliance on long-range communication and significantly cutting traffic, thus enhancing scalability as vehicle density increases.

B. Solving the Platoon Problem

Based on the MinMax K -means algorithm from [31], we introduce a new formation algorithm, CFA-FP, designed to address the platoon formation issue. Following [31], we reformulate the platoon formation objective function as the sum of variances weighted by φ_k , given by $\Omega_w = \sum_{k \in \kappa} \varphi_k^p V_k$. Here, φ_k is a weighting index that increases with the variance V_k , and the index p is a constant in the range $[0, 1)$. In this way, the expression $\sum_{k \in \kappa} \varphi_k^p V_k$ models the behavior of $\max_{k \in \kappa} V_k$, the maximum variance criterion

$$\min \sum_{k \in \kappa} \varphi_k^p V_k \quad (20)$$

$$\sum_{k \in \kappa} \varphi_k = 1 \quad \forall k \in \kappa \quad (21)$$

$$\varphi > 0, 0 \leq p < 1 \quad \forall k \in \kappa. \quad (22)$$

C. Formation Algorithm Design

We use an iterative approach. In the R th iteration, based on the weight index $\varphi_k^{(R-1)}$ and mean $m_k^{(R-1)}$ (the cluster center), each client i is allocated to the cluster that minimizes the weighted distance to $m_k^{(R-1)}$. The allocation of client i during the R th round is as follows:

$$x_i^{k(R)} = \begin{cases} 1, & k = \arg \min_{k' \in \kappa} (\varphi_k^{(R-1)})^p \|t_i - m_{k'}^{(R-1)}\|^2 \\ 0, & \text{otherwise.} \end{cases} \quad (23)$$

After updating x_i^k , the cluster center $m_{k \in \kappa}$ is recalculated using the derivative $(\partial \Omega_w / \partial \omega_k) = 0$. This derivative helps determine the cluster center k at iteration R

$$m_k^{(R)} = \frac{\sum_{i=1}^I x_i^k t_i}{\sum_{i=1}^I x_i^k}. \quad (24)$$

Next, with the client assignments $x_{i \in \kappa}^k$ and the cluster center m_k , the weight index $\varphi_k^{(R)}$ for cluster k at iteration R is updated using the derivative equation $(\partial \Omega_w / \partial \varphi_k) = 0$

$$\varphi_k^{(R)} = \frac{v_k^{\frac{1}{1-p}}}{\sum_{k'=1}^K v_{k'}^{\frac{1}{1-p}}}. \quad (25)$$

Given $0 < p < 1$, a higher variance in cluster V_k leads to an increase in the weight index φ_k . To improve MinMax

Algorithm 1 CFA-FPs

Input: Client details, number of clusters K , iteration count p , weights β , threshold ε , maximum iterations $R_{\max}$;

Output: Cluster assignments $\{x_i^k\}_{i \in I, k \in K}$;

- 1: Initialize cluster centers $\{m_k^{(0)}\}_{k \in K}$, $m_k^{(0)} = 1/M$ for each k , $R = 0$;
- 2: Compute initial time estimates $\{t_i\}_{i \in I}$;
- 3: **repeat**
- 4: $R = R + 1$
- 5: **for** $i = 1$ to I **do**
- 6: **for** $k = 1$ to K **do**
- 7: Update assignments $x_i^{k(R)}$ $i \in I$ using $\varphi_k^{(R-1)}$ and $m_k^{(R-1)}$ per Eq. (23);
- 8: **end for**
- 9: **end for**
- 10: **for** $k = 1$ to K **do**
- 11: Update centers $m_k^{(R)}$ $k \in K$ and weights $\varphi_k^{(R)}$ $k \in K$ using Eq. (26) and Eq. (24);
- 12: **end for**
- 13: Calculate stability measure $\Omega_w^{(R)}$;
- 14: **until** convergence criterion is met or $R \geq R_{\max}$
- 15: Choose a stable client in each cluster to serve as the cluster head;
- 16: **return** $\{x_i^k\}_{i \in I, k \in K}$

K -means stability, a memory mechanism is introduced to the weighting index

$$\varphi_k^{(R)} = \beta \varphi_k^{(R-1)} + (1 - \beta) \frac{v_k^{\frac{1}{1-p}}}{\sum_{k'=1}^K v_{k'}^{\frac{1}{1-p}}}, 0 < \beta < 1 \quad (26)$$

where β is a constant that controls how much the previous weighting index influences the current adjustment.

Algorithm Details: We introduce a formation algorithm, CFA-FP, in Algorithm 1 to address the formation of platoon problem. Initially, the centers $\{m_k^{(0)}\}_{k \in K}$ and the weight indices $\{\varphi_k^{(0)}\}_{k \in K}$ are set up (line 1). Next, compute the duration of one intracluster aggregation round for each client $t_{i \in I}$ (lines 7–11). Next, the centers $\{m_k^{(R)}\}_{k \in K}$, weight indices $\{\varphi_k^{(R)}\}_{k \in K}$, and client assignments $\{x_i^{k(R)}\}_{i \in I}$ are updated (lines 12–15). Then, the current $\Omega_w^{(R)}$ is determined to evaluate the existing cluster topology (line 16).

D. Optimizing Vehicular Offloading With Multiagent Reinforcement Learning

Reinforcement learning frames a natural problem as the dynamic between an agent and its surroundings. Extending this to a multiagent model, the decisions of one agent influence the environment as well as the decisions of other agents [32]. Therefore, a multiagent control scheme needs to harmonize decisions across agents to optimize system-wide task offloading. Each agent engages with its environment and exchanges information with other agents via interconnected networks, facilitating collaborative offloading management. This study

Algorithm 2 Valid Action Space Algorithm

Input: Subtask T for offloading; *serverList* includes local devices and service vehicles.

Output: Valid action space A_{valid}

```

1: Initialize  $A_{\text{valid}}$  as an empty list;
2:  $Re_{\text{required}} \leftarrow$  resources necessary for the task  $T$ ;
3: for each server in serverList do
4:    $Re_{\text{available}} \leftarrow$  available resources of server;
5:    $A_{\text{valid}} \leftarrow A_{\text{valid}} \cup \{1\}$  if  $Re_{\text{required}} \leq Re_{\text{available}}$  else
      $A_{\text{valid}} \leftarrow A_{\text{valid}} \cup \{0\}$ ;
6: end for

```

aims to delegate computing tasks to vehicles within a specified data transmission range, engaging the respective agents. Additionally, each agent exhibits Markov characteristics. At each time point t during training, examined by the agent is the state s_t , and a decision a_t is made according to the prevailing policy. Through interaction with the environment and feedback mechanisms, it seeks to optimize the expected reward, $E[\sum_{t=1}^T r_t]$. The following section constructs a multiagent Markov decision process based on the system's real structure, encompassing state space, action space, and reward function. Subsequently, an advanced reinforcement learning algorithm is introduced to tackle the offloading problem in this multiagent context.

1) *State Space*: The model for reinforcement learning is designed by incorporating the characteristics of subtasks and service vehicles in a multiagent framework. Based on the previous discussion and the symbols in Table I, the state space is specified by the tuple $s_t = (\{C_i^{\text{in}}\}_{i=1}^{TN}, \{N_i^{\text{dow}}\}_{i=1}^{TN}, \{U_i\}_{i=1}^{TN}, P, \{C_i^{\text{out}}\}_{i=1}^{TN}, \{N_i^{\text{up}}\}_{i=1}^{TN}, L, \{V_i\}_{i=1}^{n+m})$. In this context, C_i^{in} denotes the input data for the i th component of the task, while U_i indicates the CPU allocation required for it. The priority matrix for all subtasks is P , C_i^{out} indicates the output data from the i th subtask, and N_i^{up} refers to the uplink bandwidth necessary for data transfer from it. L refers to the DAG matrix depicting data dependencies among subtasks, and V_i shows the CPU consumption rate of the i th device at a specific moment t .

2) *Action Space*: To help the agent choose the right computing device for each task, the action space is defined as a one-to-one mapping between processors and tasks. For instance, $A_i = (0, 0, 1, \dots, 0, 0)$ signifies that the i th task can be assigned to the third device, meaning that this device meets the task's requirements. The action space size is $(N + 1)^K$, where the extra 1 accounts for local execution. The large action space initially complicates learning and convergence, so preprocessing can make learning more efficient and reduce iterations. Whether the i th device can handle the task is shown by $A_{\text{valid}}[i]$; it is 1 if suitable and 0 otherwise. The valid action space algorithm is described in Algorithm 2. Invalid actions in Algorithm 1 are excluded for the current task but might be considered for others.

3) *Reward Function*: The reward function is essential for guiding the agent's learning process. Different optimization goals necessitate distinct reward functions. In this study, three criteria are used to evaluate the reward for the offloading

policy: 1) average latency; 2) device load status; and 3) task priority in the multiagent setting. The goal is to reduce both energy consumption and device load while ensuring high-priority tasks have the lowest average latency. At time t_0 , the reward function is defined as

$$\begin{cases} R = -(\Omega_1(Z_{la}(t_0) + P) + \Omega_2 Z_{ls}(t_0)) \\ \Omega_1 + \Omega_2 = 1. \end{cases} \quad (27)$$

At time t_0 , the reward function incorporates average latency, device load status, and task priority, denoted as $Z_{la}(t_0)$ and $Z_{ls}(t_0)$, respectively, as detailed in (7), (11), and (13). To ensure compatibility across their varying units and scales, these values are normalized using z-score standardization. For a dataset $Z = \langle x_1, x_2, x_3, \dots, x_m \rangle$, the normalized values are represented as $\langle x'_1, x'_2, \dots, x'_m \rangle$

$$\begin{cases} \bar{x} = \frac{1}{m} \sum_{i=1}^m x_i \\ s = \sqrt{\frac{1}{m} \sum_{i=1}^m (x_i - \bar{x})^2} \\ x'_i = \frac{x_i - \bar{x}}{s}. \end{cases}$$

In reinforcement learning, the action-value function represents the expected reward of the policy π at time t [33]

$$Q^\pi(s_t, a_t) = \mathbb{E}_{r_t, s_{t+1} \sim E} [r_t(s_t, a_t) + \gamma \mathbb{E}_{a_{t+1} \sim \pi} [Q^\pi(s_{t+1}, a_{t+1})]]. \quad (28)$$

Thus, if the target policy is continuously updated, it can be represented by a function $\eta: S \rightarrow A$

$$Q^\mu(s_t, a_t) = \mathbb{E}_{r_t, s_{t+1} \sim E} [r_t(s_t, a_t) + \gamma \mathbb{E}_{a_{t+1} \sim \pi} [Q^\pi(s_{t+1}, \mu(s_{t+1}))]]. \quad (29)$$

4) *Partially Observable Markov Model*: To address the environment with incomplete state information, we employ a partially observable Markov decision process (POMDP) [34]. The POMDP is defined as a 7-tuple $(S, A, T, R, \Omega, O, \gamma)$, where S denotes the state space, A represents the action space, and Ω is the observation space. The transition probabilities between states are represented by $T: S \times A \rightarrow \pi(S)$, while $R: S \times A \rightarrow \mathbb{R}$ is the reward function. The observation probabilities are defined by $O: S \times A \rightarrow \pi(Z)$, and $\gamma \in [0, 1]$ is the discount factor. Given the continuous action space, we use the DDPG algorithm for reinforcement learning. We will present an optimization method to improve DDPG performance in the setting in the next subsection.

Algorithm 3 sets up the actor and critic networks and establishes the target network for stability. In each iterative episode, the actor-network chooses the action for task offloading, the system observes the subsequent state, and a reward is received based on the execution outcome. The experience replay buffer stores historical data and updates the network parameters through random sampling to optimize the decision policy. Through the target Q value and the soft update method, the algorithm gradually improved the task-offloading decision-making ability in the partially observable state and optimized the performance indicators, such as energy consumption, delay, and task priority.

5) *Algorithm Optimization: Optimization With an LSTM Network*: The environment's limited cognitive capabilities make it difficult to fully observe the system state at each time step, resulting in a nonstationary environment that necessitates

Algorithm 3 Partial Offloading Scheme Using DDPG With LSTM

Input: Vehicular network environment for task offloading

Output: Offloading decisions

- 1: Initialize actor network $\mu(s|\theta^\mu)$ and critic network $Q(s, a|\theta^Q)$ with parameters θ^μ, θ^Q respectively;
- 2: Initialize LSTM parameters for actor and critic networks;
- 3: Set target network parameters;
- 4: $\theta'^\mu \leftarrow \theta^\mu$
- 5: $\theta'^Q \leftarrow \theta^Q$
- 6: Set up replay buffer B and mini-batch size T ;
- 7: Set discount factor γ and update factor τ ;
- 8: **for** episode = 1 to max_episode **do**
- 9: Reset the environment and observe initial state s_0 ;
- 10: **for** $t = 1$ to T **do**
- 11: Select action a_t using $\mu(s|\theta^\mu)$;
- 12: Execute action a_t , receive reward r_t , observe next state s_{t+1}
- 13: **if** B is not full **then**
- 14: Store transition (s_t, a_t, r_t, s_{t+1}) in B
- 15: **else**
- 16: Replace a random entry in B with (s_t, a_t, r_t, s_{t+1})
- 17: Sample mini-batch from B ;
- 18: Update critic and actor networks, and target networks;
- 19: **end if**
- 20: **end for**
- 21: **end for**

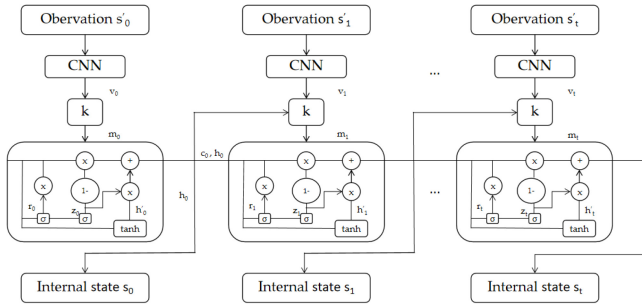


Fig. 2. LSTM-based internal state prediction.

policy adjustments. Although the DDPG algorithm excels with complete environmental observations, partial state information in the context adds complexity.

To handle the dynamic nature of the environment, we integrate LSTM into the DDPG framework for enhanced task-offloading. This addition aids in accurately estimating the state at the present moment. As illustrated in Fig. 2, the state forecasting framework includes three components: 1) a long short-term memory (LSTM) network; 2) a convolutional neural network (CNN); and 3) an attention mechanism [35]. At each moment in time t_0 , processing the current state s'_t , the CNN identifies environmental characteristics. Generating a context vector m_t from these features, the attention network follows. In conjunction with the preceding hidden state h_{t-1}

TABLE II
MAIN PARAMETERS SETTING

Parameter	Symbol	Value
Channel bandwidth	W_n	10 MHz
Noise power density	N_0	-174 dBm/Hz
Experience pool size	e	6400
Batch size	b	64
Discount factor	γ	0.99
Data upload power	P_n	0.1 W
Number of service vehicles	M	15
Data size of tasks	D_n	(0.8, 1.2) MB
Number of task vehicles	N	10
Maximum delay tolerance	T_n	1.5
Number of vehicles	K	30
Learning rate	$\alpha\beta$	0.0001, 0.001
Channel gain	h_n	$(10^{-8}, 10^{-7})$
CPU cycles of tasks	C_n	(0.1, 1.0) G
CPU frequency of the vehicle	f_n	0.5 GHz

and memory cell state c_{t-1} , this vector is fed into the LSTM, producing memory state c_t and a new hidden state h_t . To evaluate the current state s_t , the hidden state h_t is utilized.

V. SIMULATION RESULTS

Through numerical simulations in a platoon scenario, we evaluate the offloading scheme of the CFA-FP algorithm in this section. We first outline the simulation parameter settings in detail. Next, we present a comparative analysis of various parameters within the reinforcement learning algorithm. Finally, we introduce and compare evaluation metrics across three dimensions: 1) different scenarios; 2) decision algorithms; and 3) formation algorithms.

A. Simulation Setup

To conduct simulation experiments, the primary parameters for our simulations are outlined below for conducting simulation experiments. Consider a target area with two intersections, each measuring 300 m in length and width, containing 30 vehicles. The number of platoons k in the CFA-FP is determined based on the vehicle count and their spatial distribution within the area. Following formation, each group comprises 4 – 6 vehicles for further analysis. For the communication model, the channel's bandwidth is $W = 10$ MHz, noise power density is $N_0 = -174$ dBm/Hz, and the gain of the channel between the TV and serving vehicle (SV) ranges from 10^{-8} to 10^{-7} based on torque. The vehicles' transmission power is set to $P = 0.1$ W. Task data sizes vary from 0.8 MB to 1.2 MB, and CPU cycle requirements range from 0.1 to 1.0 G. The vehicle's CPU frequency is 0.5 G cycles/s. Additional details are provided in Table II.

For multiple access control, we adopt C-V2X Mode 4 (OFDMA combing time and frequency multiple access), in which vehicles autonomously select radio resources regardless of cellular coverage [16], based on their geographic location and road positioning. This implicit resource selection coordination mechanism significantly reduces packet collisions and enhances communication performance, making it particularly suitable for scenarios with varying vehicle densities and communication demands.

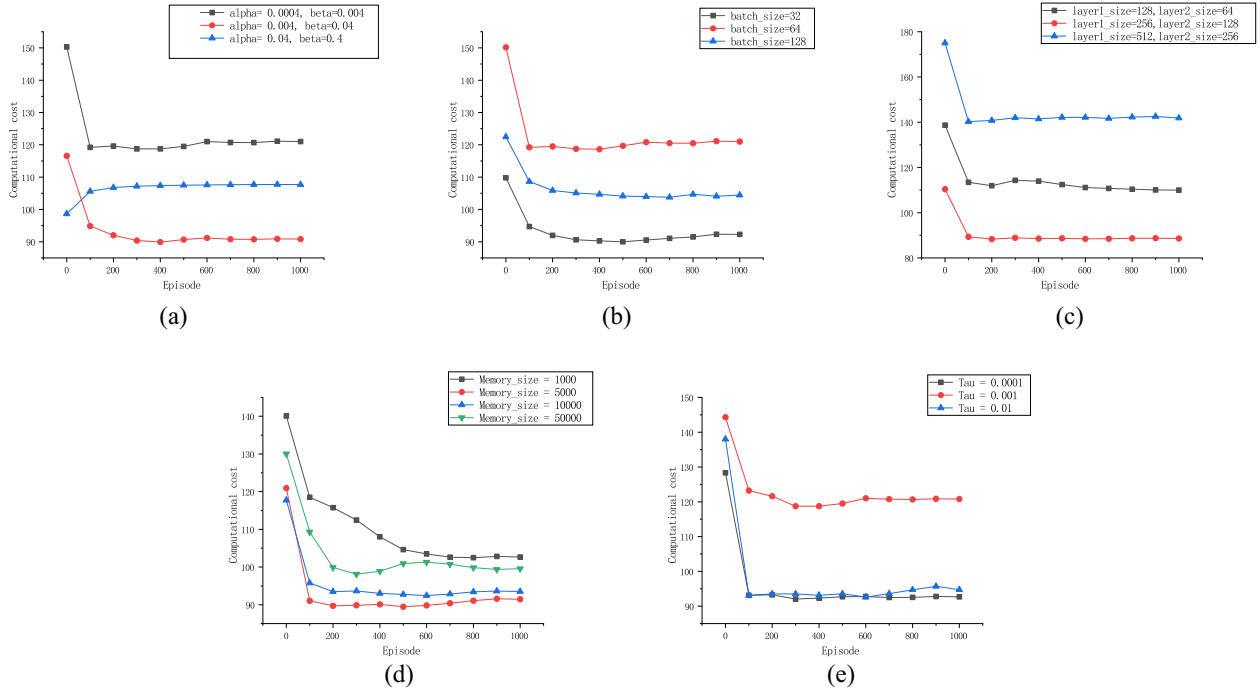


Fig. 3. Computational cost with different algorithm parameters. (a) Learning rate. (b) Batch size. (c) Network structure. (d) Experience pool size. (e) Target update parameter.

B. Comparative Experiment

1) *Parameters Settings:* We evaluated the effect of varying parameter values on the simulation results to determine the optimal settings.

- 1) The learning rate controls the step size for updating network weights during optimization. A high-learning rate can cause nonconvergence by skipping optimal points, while a low-learning rate may result in slow convergence. Thus, it is crucial to find a suitable value that strikes a balance between stability and learning speed.
- 2) The number of samples processed during each training iteration of the neural network is referred to as the batch size. While larger batches can speed up convergence, they are limited by memory capacity. Smaller batches offer more granular updates but might slow down convergence. Selecting the right batch size ensures an effective balance between convergence speed and memory utilization.
- 3) The total layers and the number of neurons within each layer of the neural network are encompassed by the network structure. Complex architectures with many layers and neurons can capture intricate data patterns but risk overfitting and increased computation. Simpler architectures may generalize better but might not capture complex data representations.
- 4) The experience pool size, or memory size, determines the amount of experience data that can be stored for replay. This setting should be carefully managed because if it is too large, it could lead to training with outdated or redundant experiences, reducing learning efficiency. Conversely, too small of a memory size might not

capture sufficient data diversity, leading to suboptimal performance.

- 5) The target update parameter, τ , dictates how quickly the target network updates to match the main network. A high- τ value synchronizes the target network more frequently, improving responsiveness but risking instability. A lower τ results in slower updates, ensuring stability but reducing responsiveness to environmental changes.

2) *Description of Comparative Results:* Analyzing these five graphs clearly shows the significant impact of different parameter combinations on reinforcement learning model performance. In Fig. 3(a), the comparison of learning rates reveals that the combination of $\alpha = 0.004$ and $\beta = 0.04$ yields the lowest and most stable computation cost, achieving rapid and stable convergence in fewer than 200 training cycles. Although $\alpha = 0.04$ and $\beta = 0.4$ also gradually converge, their final cost curve is slightly higher. The $\alpha = 0.0004$ and $\beta = 0.004$ make the model difficult to stabilize due to excessively high-learning rates, resulting in high-computation costs, highlighting that improper learning rate settings can severely impact learning performance.

In Fig. 3(b), comparing batch sizes shows that batch sizes = 32 balances training efficiency and experience pool utilization, achieving the lowest computation cost. While batch sizes = 64 quickly converge early on, it lacks sufficient samples and underutilizes the experience pool, leading to higher final costs. The batch sizes = 128 makes it difficult for the model to update in time due to the excessive batch size, resulting in relatively high costs.

In Fig. 3(c), the network structure comparison indicates that layer1 size = 256 and layer2 size = 128 provide the optimal combination, ensuring that the model quickly converges and

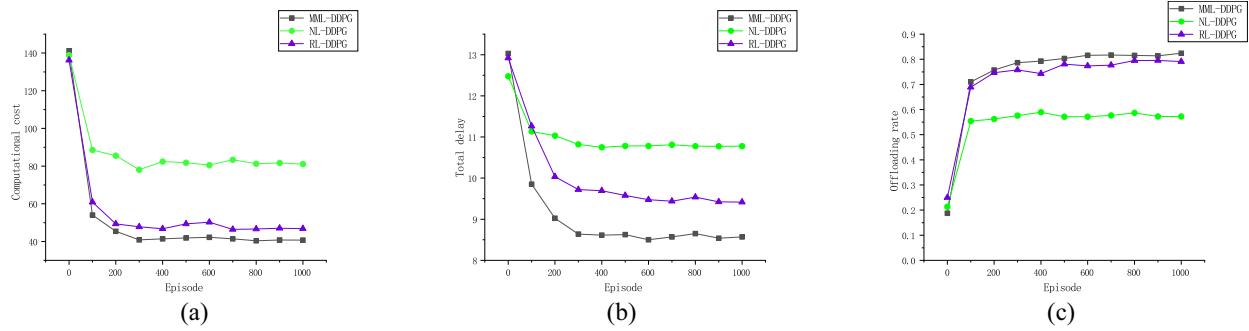


Fig. 4. Comparison of different scenarios. (a) Total computational costs for different scenarios. (b) Total delay for different scenarios. (c) Offloading rate for different scenarios.

learns complex features, achieving the lowest computation cost. The combination of layer1 size = 128 and layer2 size = 64 is too simple, failing to capture complex features, thus leading to higher computation costs. The combination of layer1 size = 512 and layer2 size = 256 is overly complex, making it prone to overfitting and resulting in higher computation costs despite its strong learning capabilities.

Fig. 3(d) demonstrates that the scale of the experience buffer determines the number of training samples available, requiring a balance between data diversity and resource consumption. The memory size = 5000 achieves the lowest computation cost, ensuring data diversity while avoiding excessive storage consumption. The memory size = 1000 lacks sufficient samples to capture enough experience data, leading to higher costs, while memory size = 50000 is rich in samples but increases storage and computation costs, resulting in slightly higher costs.

In Fig. 3(e), the τ parameter controls the synchronization speed between the target network and the main network, affecting model learning stability. The $\tau = 0.0001$ provides the optimal synchronization speed, ensuring the model can converge quickly and stably to achieve the lowest computation cost. While the $\tau = 0.0001$ keeps the cost low, it is still slightly higher relative to $\tau = 0.0001$. The $\tau = 0.01$ synchronizes too quickly to stabilize, leading to higher computation costs.

Overall, these experiments clearly illustrate the importance of optimizing parameter configuration to ensure that reinforcement learning models converge efficiently and stably, ultimately achieving optimal performance.

C. Performance Evaluation of Offloading Schemes

1) *Evaluation Metrics*: The performance metrics listed below are used to thoroughly assess the simulation outcomes.

- 1) *Computational Cost*: This metric represents the weighted sum of the cost calculated during the offloading process and the service prices paid to the service provider. By combining these two factors, the computational cost can comprehensively reflect the system's offloading performance and the training efficiency of the intelligent algorithm. A lower value indicates the higher overall efficiency of the offloading system.
- 2) *Total Delay*: This metric is the total delay from all vehicles in the grouped aggregation, indicating communication quality and the network's computational

efficiency. Lower computation latency means superior network performance and more efficient task offloading and execution. This metric is crucial to evaluate network response time and resource utilization.

- 3) *Offloading Rate*: This metric is the proportion of tasks completed within the delay constraints compared to the total number of tasks. A higher rate reflects better system stability and effective handling of delay-sensitive applications. Therefore, the offloading rate can measure the network's stability and availability and reflect the adaptability of the offloading scheme in response to dynamic environmental changes.

2) *Performance Evaluation*: We assess and compare the proposed scheme, which employs the CFA-FP algorithm with LSTM and DDPG, without RSU support, using formation techniques and DRL to optimize platoon formation and resource allocation (MML-DDPG), against three sets of baseline schemes. Group 1 compares the performance of MML-DDPG with two variations: the first is a baseline without RSU and without formation, using only the DDPG algorithm for task allocation and computational optimization (NL-DDPG), and the second includes RSU support but no formation, combining RSU and DDPG algorithms (RL-DDPG). Group 2 evaluates MML-DDPG against schemes using different reinforcement learning algorithms and task allocation methods, including a variation that combines CFA-FP with LSTM and DQN (MML-DQN) and a baseline local-only execution scenario (MM-L), where computation is restricted by local resources or delay constraints. Group 3 focuses on clustering techniques, where MML-DDPG is compared with three clustering-based approaches: one using K -means++ with LSTM and DDPG for enhanced task offloading and computational efficiency (KML-DDPG), another using standard K -means with LSTM and DDPG for basic formation and task allocation optimization (KL-DDPG), and a third using DBSCAN with LSTM and DDPG to analyze performance in varying densities (DML-DDPG).

In Fig. 4, the performance of MML-DDPG, NL-DDPG, and RL-DDPG are compared. The total computational cost of MML-DDPG decreases quickly and remains minimal, with no RSU, and CFA-FP is used to optimize resource allocation. In terms of Total delay, MML-DDPG quickly decreases and remains minimal, benefiting from efficient formation and task allocation. In terms of offloading rate, MML-DDPG rises

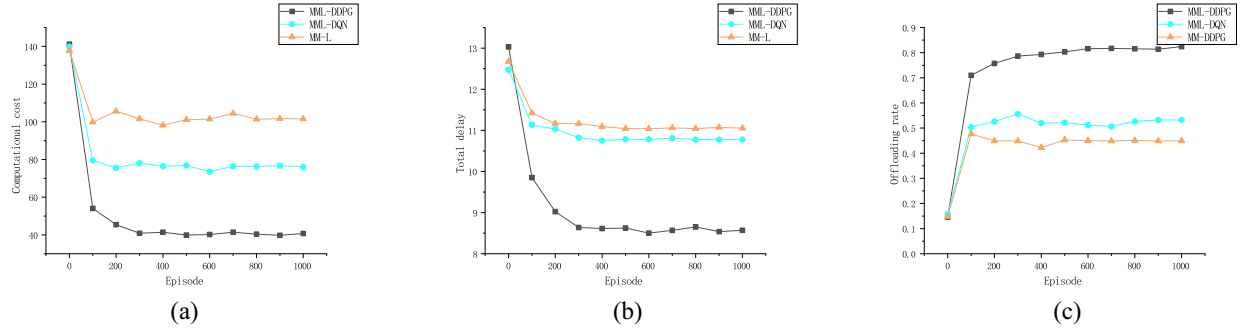


Fig. 5. Comparison of different RL algorithms. (a) Total computational costs for different RL algorithms. (b) Total delay for different RL algorithms. (c) Offloading rate for different RL algorithms.

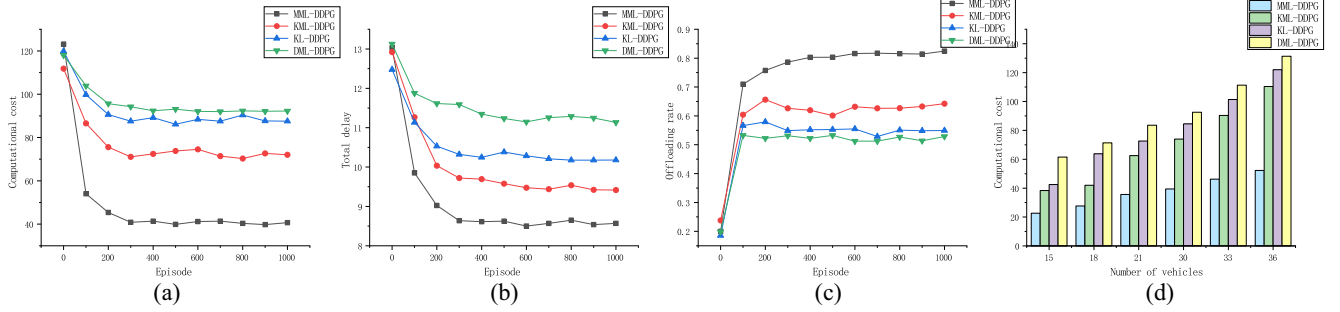


Fig. 6. (a) Total computational costs. (b) Total delay. (c) Offloading rate. (d) Number of vehicles.

faster and remains the highest, which proves that the resource utilization efficiency is the highest when there is no RSU but the formation technique is used. In contrast, NL-DDPG has high costs and delays without RSU and formation. RL-DDPG with RSU but no formation has an intermediate performance. With these comparisons, it is proved that having one platoon is optimal.

In Fig. 5, compares the performance of MML-DDPG, MML-DQN, and MM-L. Since the combination of the CFA-FP algorithm and DDPG optimizes the computational resource allocation, the total computational cost of MML-DDPG decreases quickly and remains at a minimum. In terms of delay, the initial delay of MML-DDPG is high, but it decreases rapidly and remains the lowest, which reflects the effective cooperation of LSTM and DDPG. In terms of offloading rate, MML-DDPG rises rapidly and remains the highest, indicating that its effective formation through the CFA-FP algorithm significantly optimizes task offloading and resource utilization without RSU. However, the performance of MML-DQN and MM-L is relatively poor, especially MM-L, which results in high latency and high-computational cost due to the limited local computing resources. Through these comparisons, it is proved that reinforcement learning in MML-DDPG is optimal in this case.

In Fig. 6 presents a comparative evaluation of the MML-DDPG, KML-DDPG, KL-DDPG, and DML-DDPG schemes regarding computational cost, delay, and offloading rate in a dynamic vehicular environment. The MML-DDPG scheme initially has the highest computational cost, but it swiftly decreases due to the CFA-FP algorithm's capability to optimize platoon formation and maintain a low-computational demand as the episodes progress. In terms of delay, MML-DDPG

also shows a high-starting delay, which quickly reduces and stabilizes at the lowest level, attributed to the efficient integration of LSTM and DDPG for task allocation. This low delay indicates enhanced stability in vehicle management, particularly under fluctuating vehicle numbers. As for the offloading rate, MML-DDPG quickly reaches and sustains the highest level, illustrating superior task offloading efficiency and resource optimization. Compared with KML-DDPG, KL-DDPG, and DML-DDPG, the optimized K -means-based MML-DDPG shows enhanced adaptability and stability across different vehicular densities, underscoring its effectiveness in managing computational resources and maintaining system stability. This comprehensive comparison highlights MML-DDPG's advantages in stabilizing computational performance and optimizing task management in various vehicle environments.

These comparisons demonstrate the superiority of the MML-DDPG scheme over traditional methods. Leveraging the CFA-FP algorithm for optimized platoon formation and DDPG for DRL, MML-DDPG significantly reduces the computational cost and delay while achieving the highest offloading rate. Unlike prior methods that rely heavily on RSUs, MML-DDPG ensures effective task offloading and resource allocation without RSU dependency, as evidenced by its consistently low delay and high-offloading rate across various simulation scenarios.

VI. CONCLUSION

The MML-DDPG offloading scheme proposed in this article addresses several challenges in IoV, such as the computational complexity of DRL, the difficulty of achieving efficient communication without RSU support, and the

need for careful fine-tuning of model parameters like learning rate and batch size. To overcome these issues, our approach leverages platoon-based task offloading in no-RSU scenarios, utilizing DRL combined with LSTM to manage uneven task distribution and improve offloading efficiency. The CFA-FP formation algorithm organizes vehicles into a platoon, facilitating collaboration between the platoon and surrounding vehicles on the road, thereby mitigating instability caused by intervehicle communication. Simulation results show that our method outperforms other task offloading schemes, significantly improving key performance metrics, such as computation cost, processing delay, and offloading success rate, thereby verifying the effectiveness of the MML-DDPG scheme.

REFERENCES

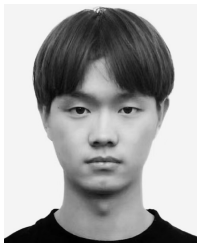
- [1] H. Ke, J. Wang, L. Deng, Y. Ge, and H. Wang, "Deep reinforcement learning-based adaptive computation offloading for MEC in heterogeneous vehicular networks," *IEEE Trans. Veh. Technol.*, vol. 69, no. 7, pp. 7916–7929, Jul. 2020.
- [2] J. Zhang, L. Zhao, K. Yu, G. Min, A. Y. Al-Dubai, and A. Y. Zomaya, "A novel federated learning scheme for generative adversarial networks," *IEEE Trans. Mobile Comput.*, vol. 23, no. 5, pp. 3633–3649, May 2024.
- [3] J. Wang et al., "Generative AI for integrated sensing and communication: Insights from the physical layer perspective," *IEEE Wireless Commun.*, vol. 31, no. 5, pp. 246–255, Oct. 2024.
- [4] L. Zhao, K. Yang, Z. Tan, X. Li, S. Sharma, and Z. Liu, "A novel cost optimization strategy for SDN-enabled UAV-assisted vehicular computation offloading," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 6, pp. 3664–3674, Jun. 2021.
- [5] P. Liu, X. Wang, A. Hawbani, B. Hua, L. Zhao, and Z. Liu, "BETA: Beacon-based traffic-aware routing in vehicular ad hoc networks," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 12, pp. 24206–24219, Dec. 2022.
- [6] J. Wang et al., "Generative artificial intelligence assisted wireless sensing: Human flow detection in practical communication environments," *IEEE J. Sel. Areas Commun.*, vol. 42, no. 10, pp. 2737–2753, Oct. 2024.
- [7] Y. Ouyang, "Task offloading algorithm of vehicle edge computing environment based on dueling-DQN," *J. Phys., Conf. Ser.*, vol. 1873, no. 1, 2021, Art. no. 12046. [Online]. Available: <https://dx.doi.org/10.1088/1742-6596/1873/1/012046>
- [8] W. Zhan et al., "Deep-reinforcement-learning-based offloading scheduling for vehicular edge computing," *IEEE Internet Things J.*, vol. 7, no. 6, pp. 5449–5465, Jun. 2020.
- [9] M. Li, J. Gao, L. Zhao, and X. Shen, "Deep reinforcement learning for collaborative edge computing in vehicular networks," *IEEE Trans. Cogn. Commun. Netw.*, vol. 6, no. 4, pp. 1122–1135, Dec. 2020.
- [10] H. Peng and X. Shen, "Multi-agent reinforcement learning based resource management in MEC and UAV-assisted vehicular networks," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 1, pp. 131–141, Jan. 2021.
- [11] M. D. Hossain et al., "Dynamic task offloading for cloud-assisted vehicular edge computing networks: A non-cooperative game theoretic approach," *Sensors*, vol. 22, no. 10, p. 3678, 2022. [Online]. Available: <https://www.mdpi.com/1424-8220/22/10/3678>
- [12] Z. Nan, S. Zhou, Y. Jia, and Z. Niu, "Joint task offloading and resource allocation for vehicular edge computing with result feedback delay," *IEEE Trans. Wireless Commun.*, vol. 22, no. 10, pp. 6547–6561, Oct. 2023.
- [13] W. Fan et al., "Joint task offloading and resource allocation for vehicular edge computing based on V2I and V2V modes," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 4, pp. 4277–4292, Apr. 2023.
- [14] N. Banitalebi, P. Azmi, N. Mokari, A. H. Arani, and H. Yanikomeroglu, "Distributed learning-based resource allocation for self-organizing C-V2X communication in cellular networks," *IEEE Open J. Commun. Soc.*, vol. 3, pp. 1719–1736, 2022.
- [15] X. Gu, J. Peng, L. Cai, X. Zhang, and Z. Huang, "Markov analysis of C-V2X resource reservation for vehicle platooning," in *Proc. IEEE 95th Veh. Technol. Conf.*, 2022, pp. 1–5.
- [16] R. Molina-Masegosa, M. Sepulcre, and J. Gozalvez, "Geo-based scheduling for C-V2X networks," *IEEE Trans. Veh. Technol.*, vol. 68, no. 9, pp. 8397–8407, Sep. 2019.
- [17] X. Deng, J. Yin, P. Guan, N. N. Xiong, L. Zhang, and S. Mumtaz, "Intelligent delay-aware partial computing task offloading for multi-user Industrial Internet of Things through edge computing," *IEEE Internet Things J.*, vol. 10, no. 4, pp. 2954–2966, Feb. 2023. [Online]. Available: <http://dx.doi.org/10.1109/jiot.2021.3123406>
- [18] I. A. Elgendy, W.-Z. Zhang, H. He, B. B. Gupta, and A. A. Abd El-Latif, "Joint computation offloading and task caching for multi-user and multi-task MEC systems: Reinforcement learning-based algorithms," *Wireless Netw.*, vol. 27, pp. 2023–2038, Apr. 2021. [Online]. Available: <http://dx.doi.org/10.1007/s11276-021-02554-w>
- [19] Z. Chen, Z. Zhou, and C. Chen, "Code caching-assisted computation offloading and resource allocation for multi-user mobile edge computing," *IEEE Trans. Netw. Service Manag.*, vol. 18, no. 4, pp. 4517–4530, Dec. 2021. [Online]. Available: <http://dx.doi.org/10.1109/tnsm.2021.3103533>
- [20] Z. Chen, L. Zhang, Y. Pei, C. Jiang, and L. Yin, "NOMA-based multi-user mobile edge computation offloading via cooperative multi-agent deep reinforcement learning," *IEEE Trans. Cogn. Commun. Netw.*, vol. 8, no. 1, pp. 350–364, Mar. 2022.
- [21] J. Peng, H. Qiu, J. Cai, W. Xu, and J. Wang, "D2D-assisted multi-user cooperative partial offloading, transmission scheduling and computation allocating for MEC," *IEEE Trans. Wireless Commun.*, vol. 20, no. 8, pp. 4858–4873, Aug. 2021. [Online]. Available: <http://dx.doi.org/10.1109/twc.2021.3062616>
- [22] A. Virdis, G. Nardini, and G. Stea, "A framework for MEC-enabled platooning," in *Proc. IEEE Wireless Commun. Netw. Conf. Workshop (WCNCW)*, 2019, pp. 1–6.
- [23] C. Yang, W. Lou, Y. Liu, and S. Xie, "Resource allocation for edge computing-based vehicle platoon on freeway: A contract-optimization approach," *IEEE Trans. Veh. Technol.*, vol. 69, no. 12, pp. 15988–16000, Dec. 2020.
- [24] J. Heinovski and F. Dressler, "Platoon formation: Optimized car to platoon assignment strategies and protocols," in *Proc. IEEE Veh. Netw. Conf. (VNC)*, 2018, pp. 1–8.
- [25] P. Wang, B. Di, H. Zhang, K. Bian, and L. Song, "Joint platoon formation and resource allocation for connected vehicles by cellular V2X communication," in *Proc. IEEE Int. Conf. Commun. (ICC)*, 2019, pp. 1–6.
- [26] A. M. I. Mahbub and A. A. Malikopoulos, "A platoon formation framework in a mixed traffic environment," *IEEE Control Syst. Lett.*, vol. 6, pp. 1370–1375, 2022.
- [27] T. Clessienne, "Opportunistic scheduling algorithm for joint-processing DL CoMP," in *Proc. IEEE 84th Veh. Technol. Conf.*, 2016, pp. 1–5.
- [28] J. E. J., M. K. Starr, and M. Zeleny, "Multiple criteria decision making," *Technometrics*, vol. 20, no. 1, p. 105, 1978. [Online]. Available: <http://dx.doi.org/10.2307/1268169>
- [29] P. H. dos Santos, S. M. Neves, D. O. Sant'Anna, C. H. de Oliveira, and H. D. Carvalho, "The analytic hierarchy process supporting decision making for sustainable development: An overview of applications," *J. Clean. Prod.*, vol. 212, pp. 119–138, Mar. 2019. [Online]. Available: <https://api.semanticscholar.org/CorpusID:158493304>
- [30] Z. Zhou et al., "Learning-based URLLC-aware task offloading for Internet of Health Things," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 2, pp. 396–410, Feb. 2021.
- [31] G. Tzortzis and A. Likas, "The MinMax k-means clustering algorithm," *Pattern Recognit.*, vol. 47, no. 7, pp. 2505–2516, 2014.
- [32] J. Liu et al., "RL/DRL meets vehicular task offloading using edge and vehicular cloudlet: A survey," *IEEE Internet Things J.*, vol. 9, no. 11, pp. 8315–8338, Jun. 2022.
- [33] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch, "Multi-agent actor-critic for mixed cooperative-competitive environments," in *Proc. 31st Int. Conf. Neural Inf. Process. Syst.*, Jun. 2017, pp. 6382–6393.
- [34] Y. Aviv and A. Pazgal, "A partially observed Markov decision process for dynamic pricing," *Manag. Sci.*, vol. 51, no. 9, pp. 1400–1416, 2005. [Online]. Available: <https://api.semanticscholar.org/CorpusID:16440232>
- [35] H. Gao, X. Wang, W. Wei, A. Al-Dulaimi, and Y. Xu, "Com-DDPG: Task offloading based on multiagent reinforcement learning for information-communication-enhanced mobile edge computing in the Internet of Vehicles," *IEEE Trans. Veh. Technol.*, vol. 73, no. 1, pp. 348–361, Jan. 2024.



Liang Zhao (Member, IEEE) received the Ph.D. degree from the School of Computing, Edinburgh Napier University, Edinburgh, U.K., in 2011.

He is a Professor with Shenyang Aerospace University, Shenyang, China. Before joining Shenyang Aerospace University, he worked as an Associate Senior Researcher with Hitachi (China) Research and Development Corporation, Beijing, China, from 2012 to 2014. He has published more than 120 papers. His research interests include ITS, VANET, WMN, and SDN.

Prof. Zhao was the recipient of the Best/Outstanding Paper Awards at 2015 IEEE IUCC, the 2020 IEEE ISPA, and the 2013 ACM MoMM. He served as the Chair for several international conferences and workshops, including the 2021 IEEE TrustCom (Program Co-Chair), the 2019 IEEE IUCC (Program Co-Chair), and the 2018–2021 NGDN Workshop (Founder). He is an Associate Editor of *Frontiers in Communications and Networking* and *Journal of Circuits Systems and Computers*. He is/has been a Guest Editor of IEEE TRANSACTIONS ON NETWORK SCIENCE AND ENGINEERING, *Journal of Computing* (Springer). He is a member of IEEE INGR SatCom and IEEE ComSoc SSC.



Yuhang Feng is currently pursuing the M.S. degree with Shenyang Aerospace University, Shenyang, China.

He is a student with Shenyang Aerospace University. His research interests include mobile edge computing, computation offloading, and resource allocation.



Ammar Hawbani received the B.S. degree in computer software and theory and the M.S. and Ph.D. degrees from the University of Science and Technology of China (USTC), Hefei, China, in 2009, 2012, and 2016, respectively.

He is a Full Professor with the School of Computer Science, Shenyang Aerospace University, Shenyang, China. Following his Ph.D. completion, he served as a Postdoctoral Researcher with the School of Computer Science and Technology, USTC from 2016 to 2019. Later, he worked as an Associate

Researcher with the School of Computer Science and Technology, USTC from 2019 to 2023. His research interests span IoT, WSNs, WBANs, WMNs, VANETs, and SDN.



Lexi Xu (Senior Member, IEEE) received the Ph.D. degree from the Queen Mary University of London, London, U.K., in 2013.

He is currently a Professor-Level Senior Engineer with the Research Institute, China United Network Communications Corporation (China Unicom). He is also a China Unicom Delegate with ITU, ETSI, 3GPP, and CCSA, and a Professor (part-time) with Beijing University of Posts and Telecommunications, Beijing, China. He has applied for more than 40 patents, published three books, and

edited five international conference proceedings. His research interests include big data, self-organizing networks, satellite systems, and radio resource management in wireless systems.

Dr. Xu served as the TPC Co-Chair for ICSINC and the Workshop/Track Chair for 5GWN, IUCC, Trustcom, SmartCNS, ICT-DM, and ISC2.



Zhi Liu (Senior Member, IEEE) received the Ph.D. degree in informatics from National Institute of Informatics, Chiyoda, Japan.

He is currently an Associate Professor with The University of Electro-Communications, Chofu, Japan. His research interest includes video network transmission and mobile-edge computing.

Dr. Liu is currently an Editorial Board Member of IEEE TRANSACTIONS ON MULTIMEDIA.



Yuanguo Bi (Member, IEEE) received the Ph.D. degree from Northeastern University, Shenyang, China, in 2010.

He joined the School of Computer Science and Engineering, Northeastern University as an Associate Professor in 2010. His current research interests focus on medium access control, QoS routing, multihop broadcast, mobility management in vehicular networks, as well as SDN enabled vehicular networks.